Security self-assessment

BuildTrust AI is a local payment simulator. This report describes implemented controls and known risks. It is not a penetration-test certificate, regulatory approval or proof that the app is ready for public hosting.

Accounts and public passwords

Private setup generates random passwords and stores only Argon2 hashes in the database. The local credential file and signing secret stay outside Git and the source archive. Login uses short-lived signed tokens kept in browser memory and linked to revocable server sessions. Logout, password changes, recovery and authority changes revoke affected sessions. Requests reload the current active user, role and project membership.

The separate public demo intentionally publishes four fictional passwords. It uses a marked data/public-demo profile and localhost port 8003. It refuses unrelated data and conflicting accounts; it never resets existing private passwords. Its settings ignore private environment/.env values. Public passwords are not a secret or a suitable configuration for an Internet-facing app. Signing secrets are still generated privately. The launcher disables paid services and optional model access.

Roles and independent review

Role	Authority	Boundary
Operator	Prepare, upload, correct and analyze	Cannot approve/execute payments
Finance Manager	Confirm procurement, check estimates/controls and approve eligible payments	Independent work only; BDT 100,000 payment ceiling
Admin	Approve checked budgets/controls, manage access and eligible payments	Cannot approve retained work they prepared/checked; configured payment ceiling
Auditor	Read assigned records, receipts and integrity results	No writes

Retained contributors survive edits and revisions. Approvals bind exact snapshots and current sources. Execution rechecks preparer, approver and executor authority. Account/source changes invalidate affected pending proposals and release reservations. Archived runs remain read-only through ordinary business endpoints.

Controls and remaining risks

Risk	Implemented protection	Remaining boundary
Password theft/login abuse	Argon2, generic errors, normalized account/IP failure buckets, expiry and Retry-After	Persistent SQLite limits and offline one-use recovery; no MFA/SSO or email recovery
Forged/stale token	Fixed signing policy, expiry, issuer/audience and current database access	Host compromise can expose signing material
Untrusted document	Private files, explicit human confirmation, escaped text and image previews	Human review may be wrong; parser vulnerabilities remain possible
Beneficiary substitution	Source-linked supplier changes with separate checking/approval	Real identity/callback evidence needs people

Risk	Implemented protection	Remaining boundary
Duplicate quantity/payment	Invoice identity, receipt allocations, payable bounds, one active proposal and unique settlement per proposal	False source records can still misstate physical delivery
Changed approval	Source/version/policy/content snapshot and current revalidation	A database owner can rewrite underlying state
Partial settlement failure	One cash/payment/ledger/allocation/receipt/audit transaction; rollback	No distributed bank transaction has been exercised
Cash shortfall	Current cash and base/delayed reserve checks	Expected inflows are assumptions
Prompt injection	Document text is data; deterministic decisions; no AI approval/payment tools	Optional explanation text may still be wrong
Resource abuse	Pre-parser body limits, bounded JSON/CSV/XLSX, PDF child time/page/pixel/output/memory/CPU limits	Not antivirus or a complete malware sandbox
Provider abuse	Fixed origins, bounded responses, stored task IDs, server-only keys, explicit paid enablement, atomic daily reservations and unknown-submission holds	Paid/live recovery needs separate verification
Spreadsheet injection	Reject imported formulas; escape exported formula-like cells	Review files in a suitable local environment
Browser disclosure	Memory-only tokens, protected previews/downloads and typed responses	XSS on an authenticated origin can access browser memory
Audit tampering	Canonical SHA-256 chain, serialized appends and first-failure verification	Full rewrite or tail removal needs an independent trusted checkpoint

Financial records and jobs

Estimates do not authorize procurement or payments. Cost and selling totals remain separate. Installments use one accepted quantity claim and exact cash reservations. Credits/corrections do not refund cash or release physical entitlement. Financial correction and accepted-cost reversal need separate linked reviews.

Jobs retain attempts, leases, heartbeats, bounded retry and restart recovery. Stale publishers are fenced. Unknown paid submissions require manual reconciliation. The supported deployment is one local process, not a distributed worker platform.

Logs contain request IDs and timing/status metadata. They omit credentials, tokens, provider keys and document contents. Numeric and upload bounds apply before expensive expansion/parsing. Hashing and parser work stay outside ordinary financial write locks, followed by current authority/version checks.

Evidence and deployment limits

See actual verification for tests, HTTP/browser workflows, parser checks and local workload results. Known-advisory scans describe published issues at their execution time; they do not prove complete security. Automated accessibility checks do not replace human assessment.

Recordings use isolated databases and addresses. Their provenance binds application source, duration and decoding. Packaging excludes private credentials, databases and uploads. The public sample-password fixture is intentionally included and labelled.

Before any wider deployment, independently review hosting, TLS, identity lifecycle, MFA, OS isolation, malware handling, backups, external audit checkpoints, monitoring, data retention and incident response. See remaining work. No real bank transfer, professional standards certification or security certification is claimed.

Twenty-item security review

Release date: October 20, 2026. “Verified” below means local automated acceptance of the named control, not independent certification. Paid-provider and professional/deployment acceptance remain separate. The four README demo passwords are intentionally public and remain unchanged.

No.	Requested check	Status and implemented control	Evidence and practical limit
1	Keep API keys out of the frontend	Verified locally: credentials stay in backend settings; provider status exposes enabled/disabled state only	Secret scan checks frontend source, built assets and archives; private-profile secrets are compared without printing them
2	Exclude .env files	Verified locally: Git, Docker and source packaging exclude .env variants and private key/credential files; empty .env.example stays public	Public-folder filename check and release archive checks
3	Scan Git history	Verified: pinned Gitleaks 8.30.1 scans every available ref, current files/build and historical release archives, including nested contents	Redacted security-secret-scan.json; pattern scans do not prove absence of every possible secret
4	Rotate exposed keys	No confirmed exposure identified by the scans; no key rotation claimed	Exact local private-value comparisons also found no matches. Any future genuine leak requires provider revocation/replacement by its owner; Git history is preserved
5	Check permissions every request	Verified locally: session, account active state, current role and membership are checked on the server	Auth and project-isolation tests; no reliance on hidden UI controls
6	Distrust client IDs, roles and prices	Verified: database identity and policy control authority; monetary calculations and payment snapshots remain server-owned	Forged role/identity tests and financial regression scenarios; authorized users can still prepare explicitly reviewed estimates
7	Isolate users' data	Verified: unrelated projects, document details and previews are unavailable to an unassigned account	Two-account tests; authorized members deliberately share project business records
8	Protect database and files	Verified locally: private runtime permissions, safe storage paths and link/traversal rejection	Windows local checks; Linux permission checks run in hosted CI. OS administrators remain trusted; files are not encrypted
9	Protect direct Admin calls	Verified: direct administration and recovery calls require current Admin authority	Operator and unassigned-account requests are denied; last-Admin safeguards remain

No.	Requested check	Status and implemented control	Evidence and practical limit
10	Test login/logout/recovery	Verified locally: persistent 30-minute sessions, server logout, current-password change and hashed single-use 15-minute recovery tokens	Auth tests and browser security workflows. Public demo passwords cannot be changed; Admin recovery needs the local owner
11	Limit login/signup/expensive calls	Verified locally: persistent account/IP admission and failure buckets, operation limits and Retry-After	Concurrency, restart and boundary tests. No public signup exists; account creation is Admin-only
12	Cap AI/provider usage	Verified locally: paid providers disabled by default; positive limits and cost bounds required; reservations commit before dispatch	Mocked budget/retry tests. Local model has daily/concurrency/token/time/response bounds. Actual paid billing still needs separately authorized live verification
13	Validate inputs on server	Verified: typed contracts, bounded decimal/JSON/import parsing, explicit versions and business validation	Input and financial suites; front-end validation is only a convenience
14	Prevent SQL injection	Verified: ORM/bound parameters and allowlisted register behavior; login injection text grants no access	Query review and adversarial login/financial tests; maintenance SQL operates on controlled schema names
15	Prevent scripts from user content	Verified locally: escaped React/text content, raster document previews, attachment downloads and same-origin CSP with no inline scripts	Browser CSP test and upload tests. Inline styles remain allowed for charts; authenticated-origin XSS would still be serious
16	Limit uploads	Verified: file allowlists/signatures, 10 MB file bound, 50 PDF pages, pre-parser body bounds and bounded workers	Existing parser/upload tests; not an antivirus or complete malware sandbox
17	Verify payments before paid-feature access	Not applicable: this app has no subscription billing or paid-feature unlock endpoint	Sandbox project settlement is revalidated on the server. No real gateway/webhook or real payment verification is claimed
18	Disable debug and redact errors	Verified locally: debug=False, generic unexpected-error responses with request IDs and fixed metadata logging	Synthetic-secret error/log test and scanner output; no request bodies, tokens or provider URLs are logged
19	Back up and restore	Verified locally: consistent database plus referenced files/signing material, checksummed private archive and isolated restore	Restore checks hashes, foreign keys and audit integrity; copied sessions/recovery tokens are revoked before fresh login. Never overwrites working data

No.	Requested check	Status and implemented control	Evidence and practical limit
20	Test two accounts and anonymous access	Verified: anonymous business endpoints reject access and separate accounts cannot cross project/file boundaries	Security tests plus role/payment browser workflows; public health, sign-in, recovery redemption and static UI are intentionally reachable

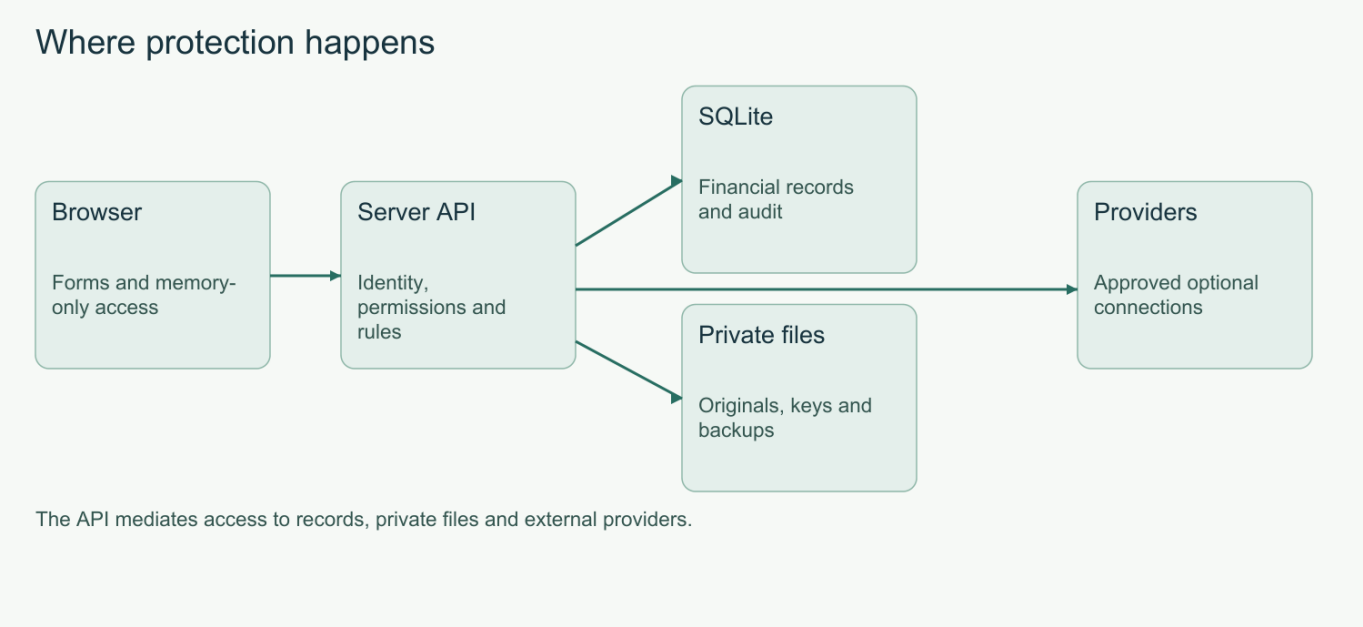
See account, recovery and backup instructions, actual release verification and the recorded test logs. Package publication is gated on the complete regression and fresh-install checks. Registration email was sent by the team; organizer confirmation and final project submission remain pending.

The 19-control security improvement checklist

Documentation-first checkpoint: MFA and new request checks are planned. Existing evidence belongs to the recorded baseline; it does not prove unfinished work.

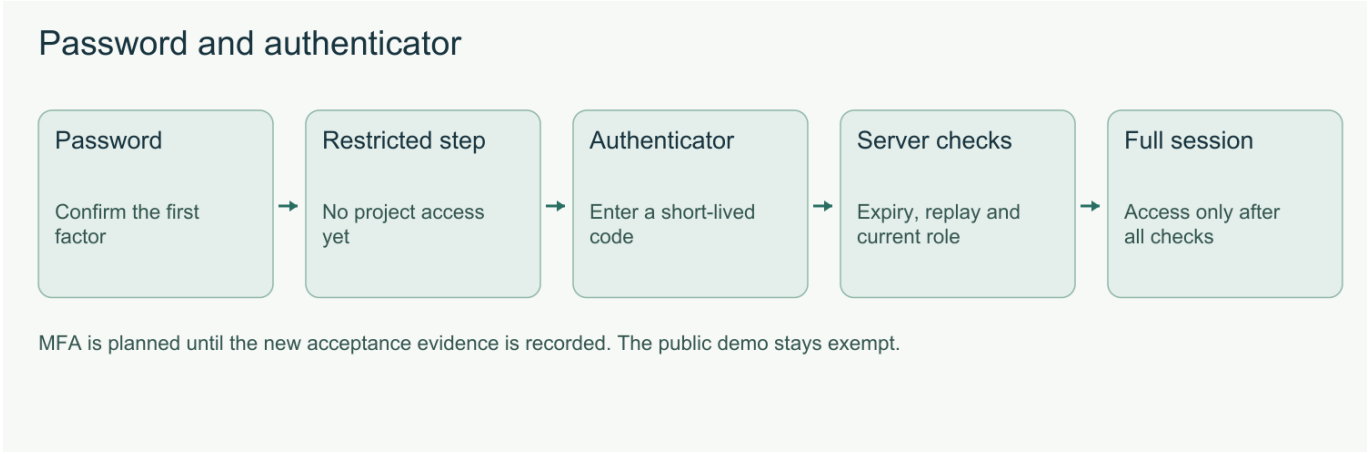
Labels: verified, implemented but unverified, planned, deferred, not applicable. The previous 20-item review remains separate.

Read the trust boundaries



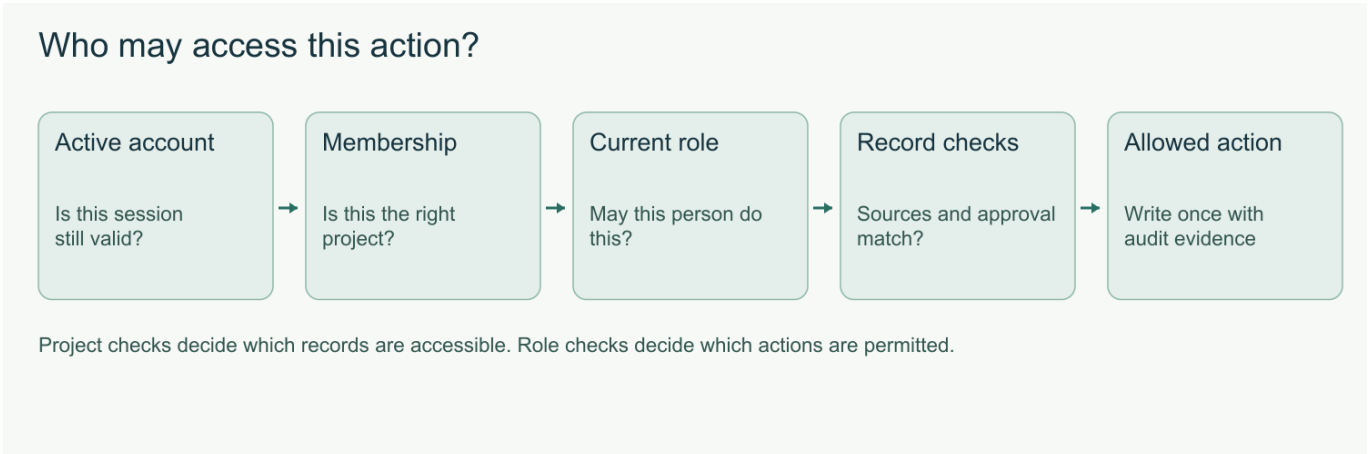
Where protection happens

The API mediates access to records, private files and external providers.



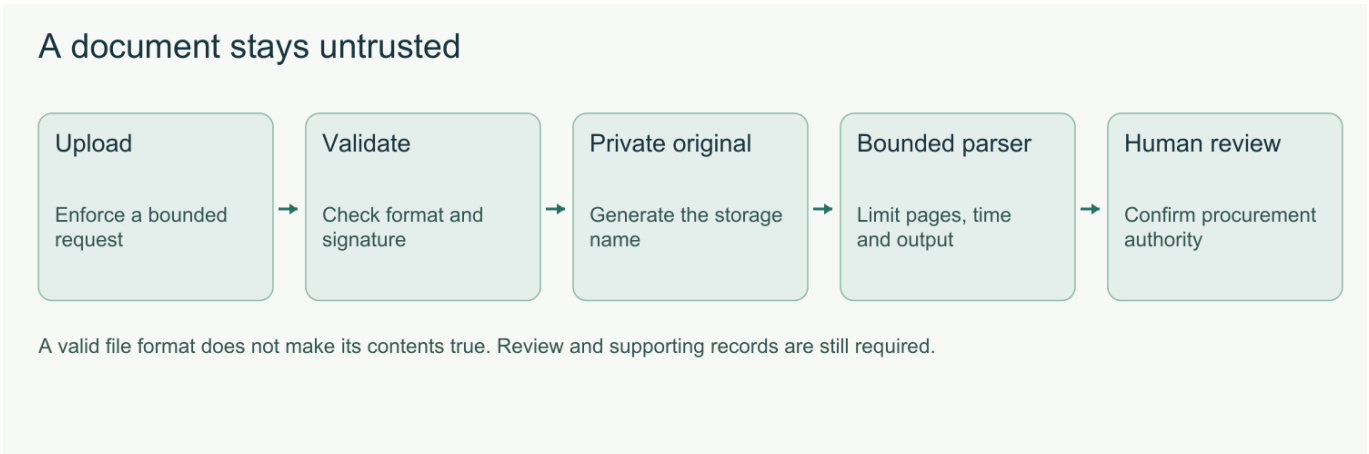
Password and authenticator

MFA is planned until the new acceptance evidence is recorded. The public demo stays exempt.



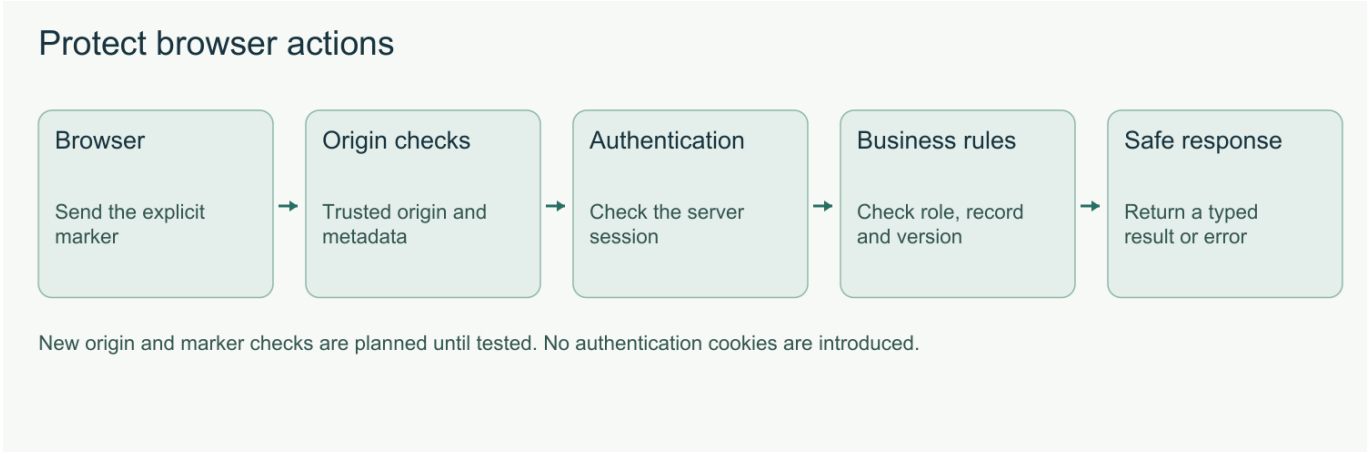
Who may access this action?

Project checks decide which records are accessible. Role checks decide which actions are permitted.



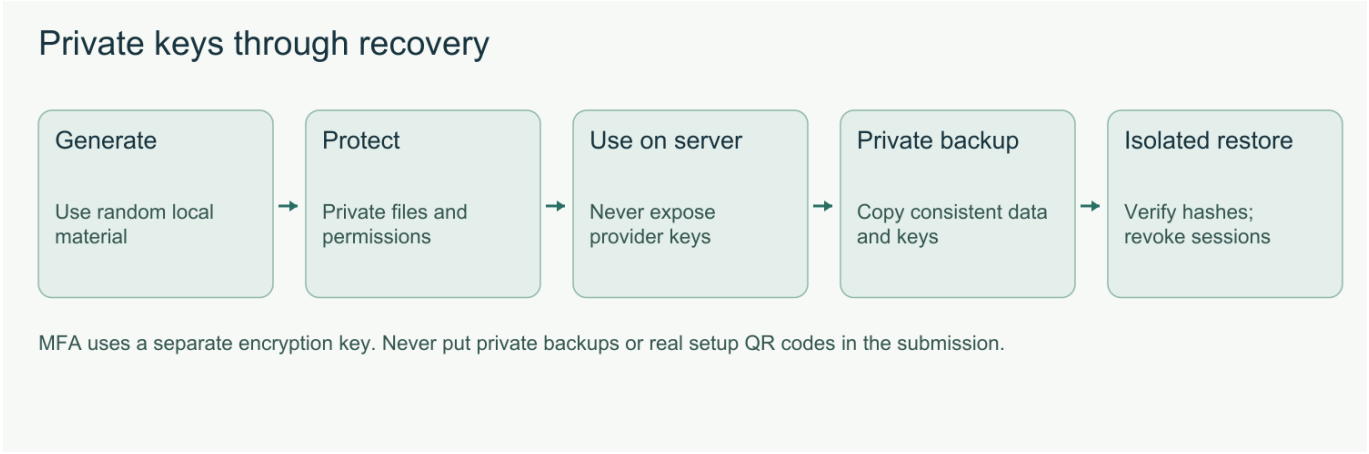
A document stays untrusted

A valid file format does not make its contents true. Review and supporting records are still required.



Protect browser actions

New origin and marker checks are planned until tested. No authentication cookies are introduced.



Private keys through recovery

MFA uses a separate encryption key. Never put private backups or real setup QR codes in the submission.

I01 - SQL injection

Purpose: Keep typed data separate from database commands.

Status: implemented but unverified.

Current baseline: ORM values are bound; maintenance queries use controlled schema names.

Acceptance and limit: Expand malicious and ordinary-input tests across registers and imports.

I02 - Cross-site scripting (XSS)

Purpose: Stop saved text from becoming a browser script.

Status: implemented but unverified.

Current baseline: React escapes text; PDF previews are images; CSP blocks inline scripts.

Acceptance and limit: Exercise stored payloads in names, quotations, links and explanations.

I03 - Cross-site request forgery (CSRF)

Purpose: Stop another website from submitting actions through a browser.

Status: planned.

Current baseline: Bearer tokens are held in memory; authentication cookies are not used.

Acceptance and limit: Add request markers, exact browser-origin checks and Fetch Metadata checks.

I04 - File upload validation

Purpose: Accept bounded supported documents and retain their evidence.

Status: verified.

Current baseline: Type/signature, file/body, page, expansion and worker limits already exist.

Acceptance and limit: Extend disguised-file and private-path regression cases; no antivirus claim.

I05 - Server-side request forgery (SSRF)

Purpose: Keep outbound connections within intended destinations.

Status: implemented but unverified.

Current baseline: Providers use fixed hosts and no redirects; Ollama is loopback-only.

Acceptance and limit: Require exact HTTPS origins, safe ports, no URL credentials or implicit proxies.

A01 - Object-level authorization

Purpose: Check which project owns every requested record.

Status: implemented but unverified.

Current baseline: Membership and resource helpers protect assigned project records.

Acceptance and limit: Expand parent/child, export, attachment, archive and publisher tests.

A02 - Rate limiting

Purpose: Bound repeated sign-ins and expensive operations.

Status: verified.

Current baseline: Persistent account/IP and operation buckets survive restarts.

Acceptance and limit: Include MFA challenges and factor-management operations.

A03 - Password hashing

Purpose: Store a slow password hash instead of the password.

Status: verified.

Current baseline: Private database passwords use Argon2; hashing stays outside financial writers.

Acceptance and limit: Retain existing hashes and generic login errors.

A04 - Multi-factor authentication (MFA)

Purpose: Require a password and an authenticator for privileged private users.

Status: planned.

Current baseline: Authenticator enrollment and verification are not implemented in the baseline.

Acceptance and limit: Require private Admin/Finance MFA; optional for other roles; public demo exempt.

A05 - Server-side permissions

Purpose: Check whether the current role may perform the action.

Status: verified.

Current baseline: The server reloads active account, role, membership and payment authority.

Acceptance and limit: Add current-role MFA assurance checks without weakening financial approval.

A06 - Native row-level security (RLS)

Purpose: Let the database itself enforce row-access policies.

Status: deferred.

Current baseline: SQLite project filters operate in the application, not as native database RLS.

Acceptance and limit: A future PostgreSQL release needs its own migration, policies and acceptance.

S01 - JWT signing secrets

Purpose: Prevent forged or stale server sessions.

Status: implemented but unverified.

Current baseline: Random private signing material, fixed JWT checks and revocable sessions exist.

Acceptance and limit: Validate key strength; keep strong existing keys and revoke old sessions for MFA.

S02 - Server-only API secrets

Purpose: Keep provider credentials outside the browser and public source.

Status: verified.

Current baseline: Backend settings, disabled paid defaults and secret scanning protect keys.

Acceptance and limit: Extend exclusions and backups for the separate MFA encryption key.

S03 - No persistent browser tokens

Purpose: Avoid leaving credentials in browser storage.

Status: verified.

Current baseline: Access tokens stay in memory; only harmless view filters use sessionStorage.

Acceptance and limit: Ensure MFA challenges, setup secrets and recovery codes also stay in memory.

C01 - Tight CORS and trusted origins

Purpose: Allow the intended local browser application only.

Status: implemented but unverified.

Current baseline: Same-origin API and the development proxy need no wildcard CORS.

Acceptance and limit: Add trusted-host and explicit per-launcher browser-origin enforcement.

C02 - Webhook signatures

Purpose: Authenticate incoming integration callbacks before processing them.

Status: not applicable.

Current baseline: There are no incoming webhook integrations or real payment callbacks.

Acceptance and limit: Keep callbacks disabled; require provider-specific verification if introduced.

C03 - Production source maps

Purpose: Do not publish browser debug maps with the running app.

Status: implemented but unverified.

Current baseline: The current production directory contains no map files.

Acceptance and limit: Set the build policy explicitly and reject map requests without SPA fallback.

C04 - Logs and lockfile hygiene

Purpose: Keep private values out of diagnostics and dependency records.

Status: verified.

Current baseline: Fixed-metadata logs, redacted errors and source/archive scans already exist.

Acceptance and limit: Add MFA values and credential-bearing dependency URLs to regression checks.

C05 - Dependency vulnerabilities

Purpose: Check published advisories and install reproducible versions.

Status: verified.

Current baseline: The recorded baseline npm and Python scans reported no known advisories.

Acceptance and limit: Rescan new dependencies; update affected compatible versions and retain locks.

Important distinctions

- XSS means content runs as a script. CSRF means a different website tries to submit an action. They require different controls.

- Project isolation is enforced by this application. Native database RLS remains deferred while SQLite is used.

- Incoming webhooks stay disabled. No real payment callback or signature-verification claim is made.

- Private Admin and Finance accounts will require MFA; the four marked public-demo accounts keep their README passwords and simple login.

- Logs and dependency lockfiles must exclude private values. A dependency scan is not an independent security assessment.

Sources: OWASP CSRF, OWASP MFA, OWASP SSRF, PyOTP.